

Lab 2

C Libraries Linking and ELF Executable Structure

Student: Preeti Pokharel Environment: Ubuntu Linux and WSL Date: October 3 2026

TABLE OF CONTENT

Task 1 Process Information Program and C Library Files

Theoretical Concept

Development Tools

Source Code

Compilation and Verification

Header Files and Library Files

Task 2 Static and Dynamic Linking

Theoretical Concept

Executable Comparison

Dynamic Section and Runtime Dependencies

Task 3 ELF Executable Structure

ELF Header

Program Headers

Knowledge Check Questions

Conclusion

List of Figures

- Figure 1 Required development tools and version information
- Figure 2 Source code for the process information program
- Figure 3 Compilation and execution of the process information program
- Figure 4 Header file locations and function declarations
- Figure 5 Shared and static C library files
- Figure 6 Dynamic and static executable size and file-type comparison
- Figure 7 Dynamic-section comparison between dynamic and static executables
- Figure 8 Runtime library dependencies resolved with ldd
- Figure 9 ELF header information for the dynamically linked executable
- Figure 10 Program headers, interpreter path, load segments, and stack permissions

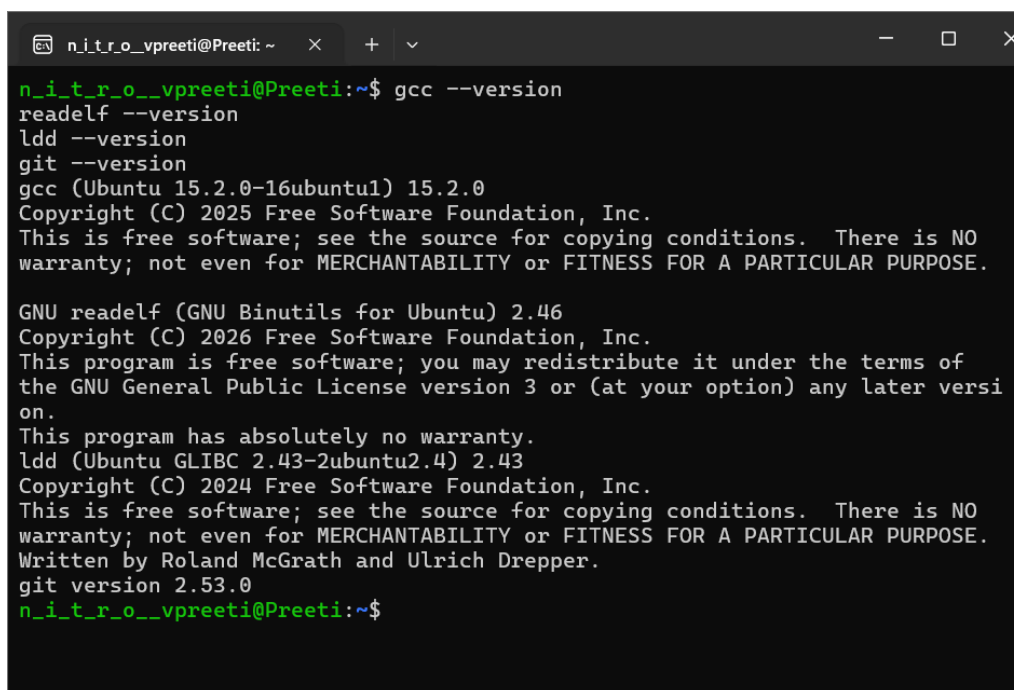
Task 1 Process Information Program and C Library Files

Theoretical Concept

A C program uses header files during compilation and precompiled libraries during linking and execution. Header files such as `stdio.h` and `unistd.h` declare functions and types so the compiler knows how they are used. The executable machine code for functions such as `printf` and `getpid` is supplied by the GNU C Library, commonly called `glibc`.

Development Tools

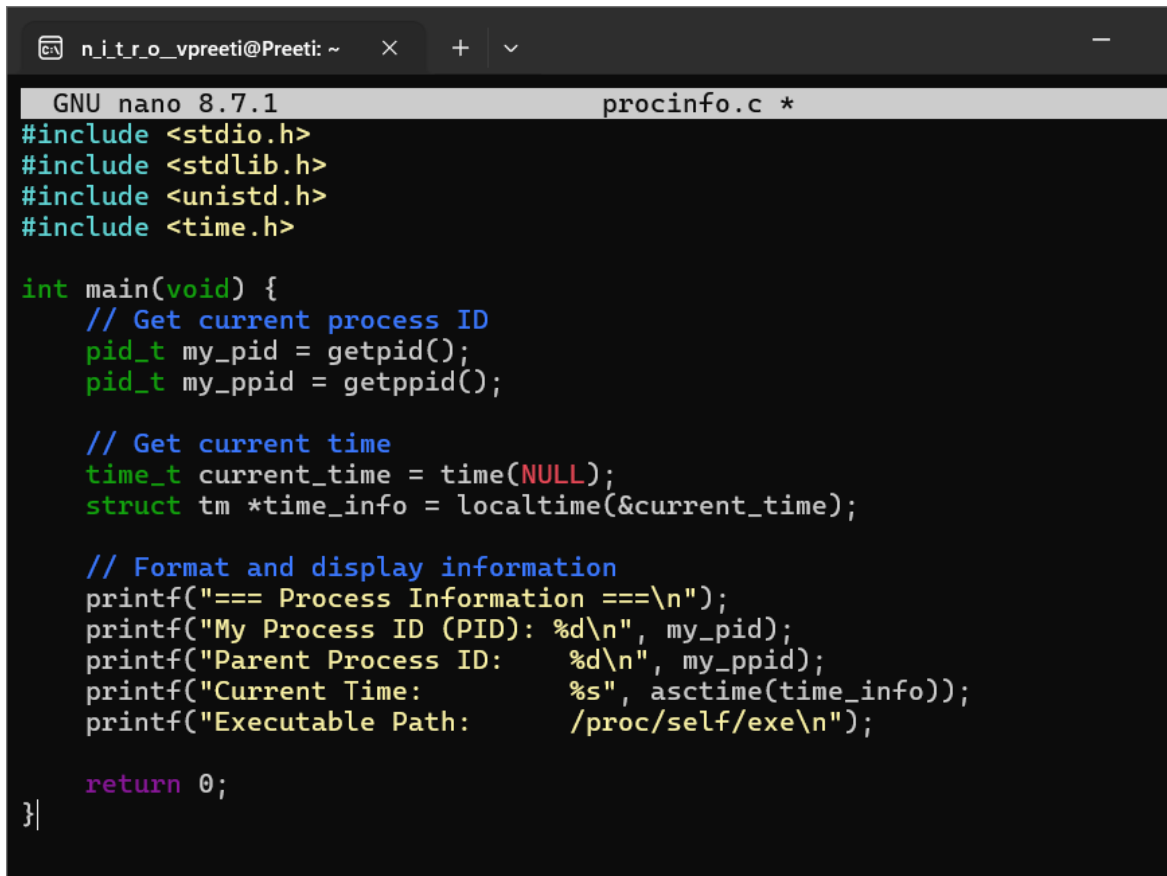
The installed versions of GCC, `readelf`, `ldd`, and `Git` were checked before the experiment. All required tools were available in Ubuntu Linux under WSL.



```
n_i_t_r_o_vpreeti@Preeti: ~  
n_i_t_r_o_vpreeti@Preeti:~$ gcc --version  
readelf --version  
ldd --version  
git --version  
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0  
Copyright (C) 2025 Free Software Foundation, Inc.  
This is free software; see the source for copying conditions. There is NO  
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  
  
GNU readelf (GNU Binutils for Ubuntu) 2.46  
Copyright (C) 2026 Free Software Foundation, Inc.  
This program is free software; you may redistribute it under the terms of  
the GNU General Public License version 3 or (at your option) any later versi  
on.  
This program has absolutely no warranty.  
ldd (Ubuntu GLIBC 2.43-2ubuntu2.4) 2.43  
Copyright (C) 2024 Free Software Foundation, Inc.  
This is free software; see the source for copying conditions. There is NO  
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  
Written by Roland McGrath and Ulrich Drepper.  
git version 2.53.0  
n_i_t_r_o_vpreeti@Preeti:~$
```

Figure 1: Required development tools and version information

Source Code



```
GNU nano 8.7.1      procinfo.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>

int main(void) {
    // Get current process ID
    pid_t my_pid = getpid();
    pid_t my_ppid = getppid();

    // Get current time
    time_t current_time = time(NULL);
    struct tm *time_info = localtime(&current_time);

    // Format and display information
    printf("=== Process Information ===\n");
    printf("My Process ID (PID): %d\n", my_pid);
    printf("Parent Process ID:      %d\n", my_ppid);
    printf("Current Time:             %s", asctime(time_info));
    printf("Executable Path:         /proc/self/exe\n");

    return 0;
}
```

Figure 2: Source code for the process information program

Compilation and Verification

The program was compiled with warning options enabled. GCC produced no warnings or errors, and the executable displayed the process identifier, parent process identifier, current time, and executable path.

```
n_i_t_r_o__vpreeti@Preeti:~$ mkdir -p ~/Lab2
cd ~/Lab2
nano procinfo.c
n_i_t_r_o__vpreeti@Preeti:~/Lab2$
n_i_t_r_o__vpreeti@Preeti:~/Lab2$ gcc -Wall -Wextra procinfo.c -o procinfo
./procinfo
=== Process Information ===
My Process ID (PID): 540
Parent Process ID: 324
Current Time: Sat Oct 3 13:23:23 2026
Executable Path: /proc/self/exe
n_i_t_r_o__vpreeti@Preeti:~/Lab2$ |
```

Figure 3: Compilation and execution of the process information program

Header File Locations and Declarations

The header files were found in /usr/include. The grep results show that stdio.h declares printf and unistd.h declares getpid. These declarations describe how the functions are called; they do not contain the complete executable implementations.

```
n_i_t_r_o__vpreeti@Preeti:~/Lab2$ ls -l /usr/include/stdio.h /usr/include/un
istd.h
-rw-r--r-- 1 root root 34258 Sep  3 14:09 /usr/include/stdio.h
-rw-r--r-- 1 root root 45053 Sep  3 14:09 /usr/include/unistd.h
n_i_t_r_o__vpreeti@Preeti:~/Lab2$ grep -n -m 1 "extern int printf" /usr/incl
ude/stdio.h
grep -n -m 1 "getpid (void)" /usr/include/unistd.h
370:extern int printf (const char *__restrict __format, ...);
650:extern __pid_t getpid (void) __THROW;
n_i_t_r_o__vpreeti@Preeti:~/Lab2$ |
```

Figure 4: Header file locations and function declarations

C Standard Library Files

The shared library `libc.so.6` and static archive `libc.a` were located on the system. The shared object supplies position-independent code that can be loaded at runtime, while the static archive contains object code that the linker can copy into an executable.

```
n_i_t_r_o__vpreeti@Preeti:~/Lab2$ ls -lh /lib/x86_64-linux-gnu/libc.so.6
ls -lh /usr/lib/x86_64-linux-gnu/libc.a
-rwxr-xr-x 1 root root 2.1M Sep  3 14:09 /lib/x86_64-linux-gnu/libc.so.6
-rw-r--r-- 1 root root 6.0M Sep  3 14:09 /usr/lib/x86_64-linux-gnu/libc.a
n_i_t_r_o__vpreeti@Preeti:~/Lab2$
```

Figure 5: Shared and static C library files

Task 1 Summary

The process-information program demonstrated the use of functions declared in standard and POSIX headers. The experiment also verified the physical separation between header declarations in `/usr/include` and compiled library code in the system library directories.

Task 2 Static and Dynamic Linking

Theoretical Concept

Linking resolves symbolic function references to executable code. With dynamic linking, the executable stores references to shared libraries that are loaded when the program starts. With static linking, the required library code is copied into the executable during compilation. Dynamic executables are normally smaller, while static executables are larger but have fewer runtime library dependencies.

Executable Comparison

```
n_i_t_r_o__vpreeti@Preeti:~/Lab2$ gcc procinfo.c -o procinfo_dynamic
gcc -static procinfo.c -o procinfo_static
ls -lh procinfo_dynamic procinfo_static
file procinfo_dynamic procinfo_static
-rwxr-xr-x 1 n_i_t_r_o__vpreeti n_i_t_r_o__vpreeti 16K Oct  3 13:26 procinf
o_dynamic
-rwxr-xr-x 1 n_i_t_r_o__vpreeti n_i_t_r_o__vpreeti 961K Oct  3 13:26 procinf
o_static
procinfo_dynamic: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), d
ynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=32
d0a17efcaccf67dd49acdc1c446aefda602252, for GNU/Linux 3.2.0, not stripped
procinfo_static: ELF 64-bit LSB executable, x86-64, version 1 (GNU/Linux),
statically linked, BuildID[sha1]=d8c62c95720ac41a0092f49f6b95953ed77e7596, f
or GNU/Linux 3.2.0, not stripped
n_i_t_r_o__vpreeti@Preeti:~/Lab2$ |
```

Figure 6: Dynamic and static executable size and file-type comparison

The dynamically linked executable was 16 KB, whereas the statically linked executable was 961 KB. The file command identified the first program as dynamically linked and the second as statically linked. The size difference occurs because the static executable contains the required library code.

Dynamic Section

The dynamic executable contains a NEEDED entry for libc.so.6. The static executable has no dynamic section because its required library code has already been included in the binary.

```
n_i_t_r_o__vpreeti@Preeti:~/Lab2$ readelf -d procinfo_dynamic | grep NEEDED
readelf -d procinfo_static
0x0000000000000001 (NEEDED)      Shared library: [libc.so.6]

There is no dynamic section in this file.
n_i_t_r_o__vpreeti@Preeti:~/Lab2$ |
```

Figure 7: Dynamic-section comparison between dynamic and static executables

Runtime Dependencies

The ldd command resolved the dynamic executable dependency to the full libc.so.6 path and displayed the dynamic linker. Running ldd on the static binary reported that it was not a dynamic executable.

```
There is no dynamic section in this file.
n_i_t_r_o__vpreeti@Preeti:~/Lab2$ ldd procinfo_dynamic
ldd procinfo_static
        linux-vdso.so.1 (0x0000793eef8b1000)
        libc.so.6 => /usr/lib/x86_64-linux-gnu/libc.so.6 (0x0000793eef600000)
)
        /lib64/ld-linux-x86-64.so.2 (0x0000793eef8b3000)
not a dynamic executable
n_i_t_r_o__vpreeti@Preeti:~/Lab2$ |
```

Figure 8: Runtime library dependencies resolved with ldd

Task 2 Summary

The results confirmed that dynamic linking produces a compact executable that depends on shared objects at runtime, while static linking creates a much larger self-contained executable with no dynamic dependency section.

Task 3 ELF Executable Structure

ELF Header

ELF stands for Executable and Linkable Format. The ELF header provides global metadata that identifies the file format, word size, byte order, executable type, processor architecture, and entry point. The inspected program is a 64-bit little-endian position-independent executable for the AMD x86-64 architecture.

```
n_i_t_r_o__vpreeti@Preeti:~/Lab2$ readelf -h procinfo_dynamic | grep -E "Mag
ic:|Class:|Data:|Type:|Machine:|Entry point"
  Magic:      7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                               ELF64
  Data:                               2's complement, little endian
  Type:                               DYN (Position-Independent Executable fi
le)
  Machine:                               Advanced Micro Devices X86-64
  Entry point address:                   0x1140
n_i_t_r_o__vpreeti@Preeti:~/Lab2$ |
```

Figure 9: ELF header information for the dynamically linked executable

ELF Header Command

```
readelf -h procinfo_dynamic
```

The Magic field begins with 7f 45 4c 46, which identifies an ELF file. The entry point address is where execution begins after the loader has prepared the process.

Program Headers

Program headers tell the operating-system loader which parts of the executable must be mapped into memory. INTERP identifies the program interpreter, or dynamic linker. LOAD entries describe readable, executable, and writable memory segments. GNU_STACK shows that the stack is readable and writable but not executable, which is the secure default.

```
n_i_t_r_o__vpreeti@Preeti:~/Lab2$ readelf -W -l procinfo_dynamic | grep -E "
INTERP|LOAD|GNU_STACK|Requesting program interpreter"
  INTERP          0x000374 0x0000000000000374 0x0000000000000374 0x00001c 0x0
0001c R          0x1
      [Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
  LOAD           0x000000 0x0000000000000000 0x0000000000000000 0x00007b8 0x0
007b8 R          0x1000
  LOAD           0x001000 0x00000000000001000 0x00000000000001000 0x0000311 0x0
00311 R E 0x1000
  LOAD           0x002000 0x00000000000002000 0x00000000000002000 0x00001d0 0x0
001d0 R          0x1000
  LOAD           0x002d80 0x00000000000003d80 0x00000000000003d80 0x0000290 0x0
00298 RW 0x1000
  GNU_STACK      0x000000 0x0000000000000000 0x0000000000000000 0x0000000 0x0
00000 RW 0x10
n_i_t_r_o__vpreeti@Preeti:~/Lab2$ |
```

Figure 10: Program headers, interpreter path, load segments, and stack permissions

Program Header Command

```
readelf -l procinfo_dynamic
```

The interpreter path was `/lib64/ld-linux-x86-64.so.2`. The R E segment contains executable instructions, the R segment contains read-only data, and the RW segment contains writable data.

Question 1 Fundamental difference between a C header and a shared library

A header file contains declarations, type definitions, structures, and macros used by the compiler. A shared library contains compiled machine code that is loaded and linked at runtime. For example, `stdio.h` declares `printf`, while `libc.so.6` contains its compiled implementation.

Question 2 Static and dynamic linking

Static linking copies required library code into the executable at build time, producing a larger and more self-contained program. Dynamic linking stores references to shared libraries, producing a smaller executable whose dependencies are loaded at runtime.

Question 3 ELF header and commands used to inspect headers

The ELF header is the metadata at the beginning of an ELF file. It identifies the file as ELF and records information such as class, byte order, file type, target machine, and entry point. The command `readelf -h <file>` displays the ELF header, while `readelf -l <file>` displays the program headers.

Question 4 Meaning of a bookmark with an example

A bookmark is a saved reference to a particular location so it can be found again quickly. For example, a browser bookmark can save the URL of the GitHub repository containing this lab. In a document, a bookmark can mark the Knowledge Check section so a link can jump directly to it.

Question 5 Documentation and GitHub submission

<https://github.com/Preetipokharel/Preetipokharel-operatingSystem-batch38.git>

Conclusion

This lab demonstrated how a C program uses declarations from header files and implementations from the GNU C Library. The experiment located the relevant header and library files, compiled and ran a process-information program, compared static and dynamic linking, and inspected the resulting ELF executable. The `readelf` and `ldd` results showed how the operating system identifies the executable format, maps memory segments, loads the dynamic linker, and resolves the `libc` dependency. The findings confirm that dynamic linking reduces executable size through shared runtime code, while static linking increases file size in exchange for independence from shared-library loading.